# Comparative Analysis of State Representations for Deep Q-Learning in Snake Game

## Problem Statement

Playing Snake effectively requires strategic planning that traditional programming struggles with. While reinforcement learning offers promise, the choice of state representation significantly impacts learning efficiency and final performance. This project investigates which state representation yields the most effective and sample- efficient learning for Snake.

## Research Questions

1. RQ1(Primary): How do different state representations (feature-based vs.CNN- based) affect the performance and training efficiency of DQN agents in Snake?
2. RQ2(Baseline): How does DQN performance compare to human baseline performance?
3. RQ3(Progression): What qualitative strategies emerge at different training stages, and how do they differ between representations?

## Methodology: Experimental Comparative Study

Research Design: A controlled experiment comparing two DQN variants with different state representations against established baselines.

Compared Agents:

1. Human Baseline: Average performance from 5 human players (10 games each)

2. Random Agent: Stochastic policy (control baseline)

3. DQN-Feature: Dense neural network processing engineered features

4. DQN-CNN: Convolutional neural network processing grid images

State Representations:

- Feature-based (DQN-Feature): Vector of 10 features:

text

[food_dx, food_dy, # Relative food position (normalized) danger_left, danger_front, danger_right, # Binary collision sensors direction_left, direction_right, direction_up, direction_down, # One-hot snake_length_normalized] # Length / max_possible_length

- CNN-based (DQN-CNN): 30×30×3 grid where:
  - Channel 0: Snake body (1.0 for body, 0.0 otherwise)
  - Channel 1: Snake head (1.0 for head)
  - Channel 2: Food position (1.0 for food)

Training Protocol:

- Computational Budget: 10,000 training episodes per agent (feasible on CPU)
- Evaluation: 100 episodes every 1,000 training episodes (ε=0)
- Random Seeds: 3 different seeds for statistical robustness
- Hardware Assumption: Standard laptop CPU (no GPU acceleration)

Justified Hyperparameters:

- Batch Size: 64 (common for DQN, balances stability and efficiency)
- Replay Buffer: 10,000 experiences (standard for medium-sized problems)
- Target Update: Every 100 episodes (moderate stability-memory trade-off)
- Learning Rate: 0.001 (Adam default, works well for many RL problems)
- γ (Discount): 0.95 (values short-term rewards while considering medium-term)
- ε-decay: 1.0 → 0.1 over first 5,000 episodes, then fixed (standard schedule)

Metrics:

1. Primary: Mean score over 100 evaluation episodes

2. Secondary:

- Training efficiency: Episodes to reach human baseline performance
- Maximum score achieved
- Survival time (moves per game)
- Qualitative strategy analysis (via gameplay recordings)

Human Baseline Protocol:

- Recruit 5 participants with basic Snake experience
- Each plays 10 games with the same interface agents use
- Record average and distribution of scores
- This provides a realistic performance target and interpretable benchmark

## Expected Results

1. Performanc eHierarchy: DQN - CNN > DQN - Feature > Random, with DQN agents approaching but not exceeding expert human performance within the training budget.
2. Training Efficiency: DQN - Feature will learn faster initially but plateau earlier; DQN-CNN will have slower start but higher final performance.

3. Human Comparison: We expect DQN agents to surpass average human performance but remain below expert human play due to limited training budget.
4. Emergent Behaviors: DQN-CNN will develop more spatially-aware strategies; DQN-Feature will exhibit more predictable, rule-like behaviors.

## Contingency Plans

1. If training is too slow: Reduce grid size to 20x20 for faster convergence while maintaining problem complexity
2. If DQN struggles: Implement simpler Double DQN instead of vanillaDQN
3. If human baselining is difficult: Use pre-recorded human game play data or simplify to single experienced player

# Environment and Code Base

The Snake game environment used in this project is based on code provided by Henrik Strøm. The implementation is written in Python and uses the PyGame library for rendering and interaction. The original game logic defines the grid-based environment, snake movement, food placement, and termination conditions.

To support reinforcement learning without rewriting or altering the underlying game mechanics, the original implementation is preserved as much as possible. In particular the Vector, Snake and Food classes remain unchanged. Minor adaptations were made to the SnakeGame class to allow programmatic control rather than keyboard input.

The original game loop, implemented in the run() method of SnakeGame, contains a continuous while-loop and keyboard-based control, which is unsuitable for reinforcement learning. Instead of modifying this loop, it is bypassed entirely. The game is driven externally through a custom environment wrapper, ensuring that the original rules and dynamics of the game remain intact.

## Environment Wrapper

To interface the Snake game with reinforcement learning agents, a custom environment wrapper names SnakeEnvironment was implemented. This wrapper is responsible for managing episodes, applying agent actions, computing rewards and exposing state representations.

The __init__() method initializes the environment by creating instances of the game, snake and food objects, while also tracking episode termination. A  new episode is started through the reset() method, which reinitializes the snake and food positions, resets the score and returns the initial state.

Agent actions are applied using the private method _apply_action(action), which replaces keyboard input with discrete integer actions corresponding to movement directions. This

abstraction allows agents to interact with the environment using a fixed action space without modifying the game logic.

The core reinforcement learning interaction is implemented in the step(action) method. Each call to this method advances the environment by exactly one timestep, applies the selected action, updates the game state, computes the reward and checks for terminal conditions. This design closely follows the standard reinforcement learning environment interface.

To validate the correctness of the environment wrapper, a random agent was implemented and used for testing. The random agent confirmed that the snake moves correctly, food spawns as expected, episodes terminate properly, and no runtime errors occur. This verification step ensured that agents can interact safely with the environment before introducing learning algorithms.

```
# Test with a random agent
env = SnakeEnvironment(render=True)
state = env.reset()

done = False
while not done:
    action = np.random.randint(0, 4)
    state, reward, done = env.step(action)

print("Episode finished")
```

### Actions and rewards

The action space and reward function were defined once and kept fixed throughout all experiments, this is a critical design decision, as altering actions or rewards during experimentation, would invalidate comparisons between agents.

The action space consists of four discrete actions corresponding to movement directions, left, right, up, and down. All agents, including the random agent, DQN-based agents and human players, interact with the environment using this same action space.

A sparse reward function was employed to minimize reward shaping and bias. Agents receive a reward of +1 for consuming food, -1 for terminal collisions, wall of self-collision, and 0 otherwise. This minimal reward structure ensures that performance differences primarily arise from differences in state representation and learning capability rather than handcrafted incentives.

## Feature-Based State Representation

The feature-based DQN agent operates on a compact, manually engineered state representation designed to capture essential information about the game environment. The state is represented as a fixed length vector of ten features.

The first two features encode the relative position of the food with respect to the snake's head, normalized by the grid size. Three binary danger sensors indicate whether moving left, forward, or right relative to the snake's current direction would result in a collision. Four additional features represent the current movement direction of the snake using a one-hot encoding. Finally, the snake's current length is normalized by a predefined maximum length to provide a measure of progress.

This representation significantly reduces the dimensionality of the state space, enabling faster learning and improved sample efficiency. However, it also limits the agent's ability to reason about global spatial structure, as information about the full grid layout is not explicitly available.

## CNN-Based State Representation

In contract to the feature-based representation, the CNN-based DQN agent operates on a spatial representation of the entire game grid. The environment state is encoded as a three-channel grid of size 30 x 30, corresponding to the dimensions of the Snake game board.

Each channel encodes a distinct element of the environment. The first channel represents the snake's body, where grid cells occupied by the snake are assigned a value of 1.0 and all other cells are 0.0. The second channel marks the snake's head position using the same binary encoding. The third channel represents the food location. This separation of entities into individual channels allows the neural network to distinguish between different game components while preserving their spatial relationships.

This grid-based representation provides the agent with complete spatial information about environment, enabling it to reason about distances, obstacles, and available free space. However, it also results in a significantly higher-dimensional state space compared to the feature-based approach.

Convolutional neural networks (CNNs) are well suited for this type of input, as they are designed to exploit local spatial correlations and translational invariance. By applying shared convolutional filters across the grid, the CNN-based agent can detect spatial patterns such as walls, body segments and food locations without requiring manual feature engineering. This representation trades sample efficiency for expressive power, allowing the agent to potentially learn more complex and globally coherent strategies.

## Neural Network Architectures

Two different neural network architectures are employed in this study, corresponding to the two state representations.

The DQN-Feature agent uses a feed-forward multilayer perception (MLP) that processes the low dimensional feature vector. The network consists of fully connected layers with rectified linear unit (ReLu) activations and outputs Q-values for the four available actions. This architecture is computationally efficient and well suited for compact, engineered features.

The DQN-CNN agent replaces the MLP with a convolutional neural network. The CNN processes the three-channel grid input using stacked convolutional layers followed by fully connected layers that output action Q-values. Convolutional layers allow the network to learn spatial filters that capture local patterns, while the fully connected layers integrate this information to support decision making.

Both agents share identical output dimensions, optimization methods, and training hyperparameters. This ensures that observed performance differences are attributable to the state representation and network architecture rather than differences in training setup.

## Training Procedure

Both DQN agents are trained using the standard Deep Q-Learning algorithm with experience replay and a target network for stability. At each time step, the agent selects an action using an epsilon-greedy policy, where a random action is chosen with probability ε is linearly decayed from 1.0 to 0.1 over the first 5,000 training episodes and held constant thereafter.

Transitions consisting of the current state, selected action, received reward, next state, and termination flag are stored in a replay buffer with a fixed capacity of 10,000 experiences. During training, minibatches are sampled uniformly form this buffer to update the policy network. This mechanism breaks temporal correlations between consecutive experiences and improves learning stability.

To further stabilize training, a separate target network is used to compute target Q-values for the Bellman update. The target network parameters are periodically updated by copying the weights of the policy network every 100 episodes. This delayed update strategy reduces divergence during training.

Training is conducted for 10,000 episodes per agent and repeated across three different random seeds to ensure statistical robustness. Performance is evaluated every 1,000 episodes over 100 evaluation episodes with exploration disabled ($\varepsilon = 0$), allowing for consistent comparison across agents and training stages.

## Training Progression and Extended Training

To analyze the effect of training duration on learning performance, the CNN-based DQN agent was trained incrementally for 1,000, 5,000, and 10,000 episodes using identical hyperparameters and reward structure. This progressive training setup allows for an examination of both early learning behavior and long-term convergence characteristics under a fixed computational budget.

After 1,000 training episodes, the CNN-based agent exhibited performance close to that of a random baseline, with no consistent increase in average score or survival time. While exploratory behavior dominated during this phase due to a high ε value, no meaningful policy improvements were observed.

Extending training to 5,000 episodes, during which ε decayed linearly to its minimum value of 0.1, resulted in stable but limited performance gains. Although the agent demonstrated slightly longer survival times in isolated episodes, the average score remained low and highly variable across evaluation runs. No clear upward learning trend emerged during this phase.

To determine whether the lack of improvement was caused by insufficient training time, the agent was further trained for a total of 10,000 episodes. During this extended training period, exploration was minimal and the agent primarily exploited its learned policy. Despite this, performance did not improve substantially, and evaluation results remained near random performance levels across all random seeds. These results indicate that the CNN-based DQN converged to a suboptimal policy under the given training constraints.

To assess whether the observed performance limitations were due to representational capacity rather than the grid-based state itself, an additional experiment with a higher-capacity CNN was conducted.

Despite increasing the depth and width of the convolutional network, the CNN-based DQN failed to consistently learn effective food-seeking strategies within 10,000 episodes on CPU. The agent frequently converged to local movement patterns such as looping or wall-following, indicating that sample efficiency — rather than representational capacity — was the primary limiting factor

# Results (Quantitative Results)

# Discussion

**RQ1: Effect of State Representation**

The results demonstrate that feature-based representations substantially outperform CNN-based grid representations under a limited training budget and CPU-only constraints. While the CNN agent had access to complete spatial information, it failed to translate this into effective policies, highlighting the importance of sample efficiency over representational completeness in this domain.

**RQ2: Comparison with Human Baseline**

The feature-based DQN approached average human performance within the training budget, while the CNN-based agent remained far below human baseline levels. This suggests that engineered features can encode domain knowledge that aligns more closely with human strategies.

**RQ3: Strategy Emergence**

Feature-based agents developed consistent food-seeking behaviors, whereas CNN-based agents converged to survival-oriented but goal-ineffective strategies such as looping and wall-following.

# Limitations & Future Work

## Limitations

- Limited training budget due to CPU-only hardware
- No GPU acceleration for CNNs
- Vanilla DQN only (no Double DQN / Dueling DQN)
- Fixed reward structure

## Future Work

- GPU-accelerated CNN training
- Frame stacking for temporal awareness
- Double DQN or Rainbow DQN
- Curriculum learning or shaped rewards
- Larger replay buffers

# Conclusion

This study compared feature-based and CNN-based state representations for Deep Q-Learning in the Snake game. Contrary to expectations, the feature-based DQN consistently outperformed the CNN-based agent across all training durations under CPU constraints. These findings suggest that compact, engineered representations can offer superior sample efficiency in environments with limited training budgets. While CNNs provide expressive spatial representations, their effectiveness is constrained by computational resources and

exploration efficiency. Overall, this work highlights the critical role of state representation in reinforcement learning performance.